PROJECT OVERVIEW

AgentGate

An approval gateway for AI agent tool calls: safe actions run immediately, risky ones wait for a human, forbidden ones are refused, and everything is recorded.

Raihan Evanza · Fullstack & Mobile Developer · portfolio-raihan-evanza.vercel.app

ROLE

Personal project · system design, backend, web, mobile, deployment

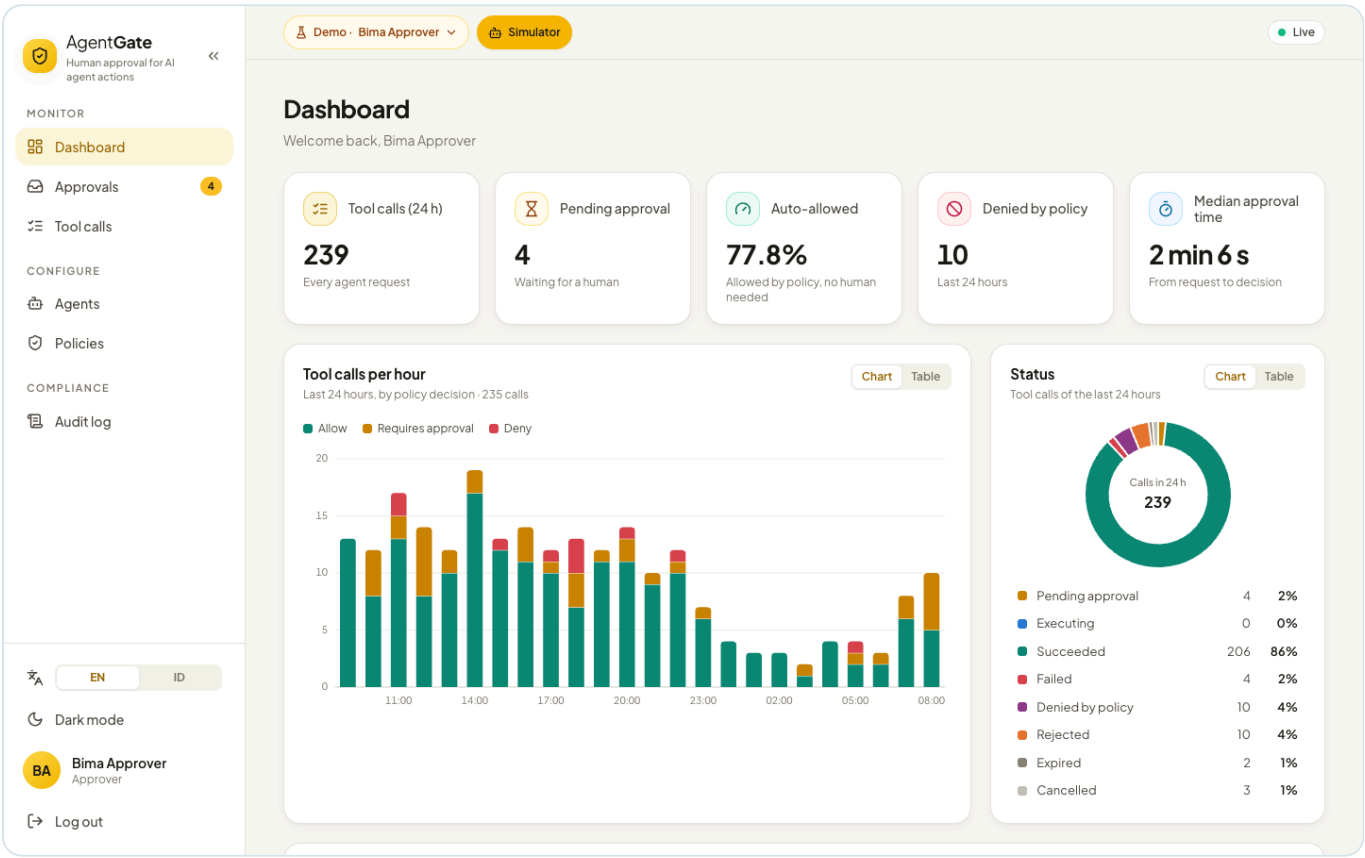
PLATFORMS

Web (console) · Android & iOS (approver app) · API gateway for agents

LINKS

Live demo: raihan-evanza-agentgate.vercel.app/?as=approver

Go 1.25 · Gin · GORM + pgx · PostgreSQL 17 · Server-Sent Events · React 18 · TypeScript · Vite · Redux Toolkit (RTK Query) · Tailwind CSS · React Native 0.87



OVERVIEW

AI agents such as support bots, data assistants, and ops bots are increasingly given tools with real side effects: refunds, database writes, emails, even production deployments. Without a control layer, the choice is either full access or no agents at all.

AgentGate sits between the agent and its tools. Every call is evaluated against a per-tool policy: safe actions run immediately (HTTP 200), risky ones wait for an approver (202), and forbidden ones are refused (403). Approvers decide from the web console or a mobile app, an agent's owner may not approve that agent's actions, and every step goes into an audit log.

The result is a Go + PostgreSQL backend with two APIs (a gateway for agents, a console for people), a React web console with real-time updates, and a React Native app for approvers. The web demo runs entirely in the browser, agent simulator included, so it can be tried without a server.

KEY FEATURES

Gateway for agents

Agents call POST /gw/v1/calls with an API key and get 200 (ran), 202 (waiting for approval), or 403 (denied by policy). After a 202, the agent waits for the outcome with long polling of up to 30 seconds per request.

Per-tool policy engine

Each tool has a risk level, a default effect, an approval window, and ordered rules with conditions on the arguments, such as amount > 10,000,000 → Deny or sql matches drop|truncate → Deny. The first rule whose conditions all hold decides.

Approvals inbox

Waiting calls are ordered by closest expiry, with a live countdown. Approvers can approve or reject; a rejection requires a note, which is passed back to the agent.

Separation of duties

An agent's owner cannot approve or reject that agent's calls, even as an admin. The Approve button is still shown but locked, with a note that someone else has to decide.

Real time, no refresh

Server-Sent Events push new calls and status changes to the console: the pending badge, lists, the detail page, and the dashboard all update. A Live indicator in the header shows the connection state.

Dashboard and audit log

The dashboard shows 24-hour tool calls, the share allowed automatically, policy denials, median approval time, an hourly chart by decision, and a status donut, each switchable to a table. The audit log records sign-ins, decisions, and agent changes with old/new values and IP address.

Agents and API keys

Admins register agents and receive an API key that is shown only once; only its SHA-256 hash is stored. Keys can be rotated (the old key stops working at once) and agents can be suspended.

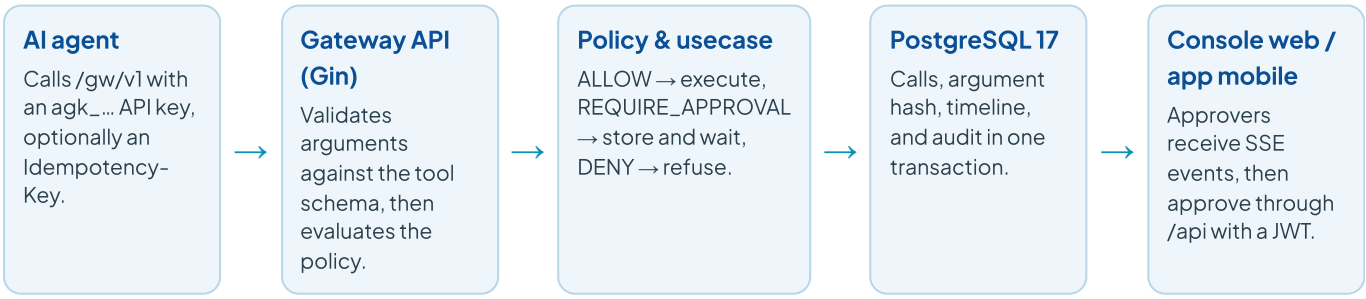
Mobile app for approvers

A React Native app (Android + iOS) with the pending inbox, badge and countdown, approve/reject buttons, and an activity feed, so decisions can be made from a phone.

Agent simulator, two languages, dark mode

In the demo, the Simulator panel calls the gateway as an agent with six scenarios (order lookup, small/large refund, DROP TABLE, production deploy, failing refund) and shows the HTTP response. The interface is available in Indonesian and English, with a light/dark theme.

ARCHITECTURE & DATA FLOW



- A single Go binary serves two APIs: the Gateway API for agents (API key) and the Console API for people (JWT). Business rules live in the usecase and policy layers; controllers only parse requests and send responses.
- Once an approver approves, the status moves to EXECUTING, the tool runs with the stored arguments, and the result wakes the agent's long poll and reaches the console over SSE.
- Background workers expire approvals that pass their deadline, and calls left in EXECUTING by a restart become FAILED instead of being re-run automatically.

- In the Vercel demo, the whole backend (policy engine, sandbox tools, expiry, real-time events) is replaced by an in-browser adapter that keeps data in localStorage.

STACK BY LAYER

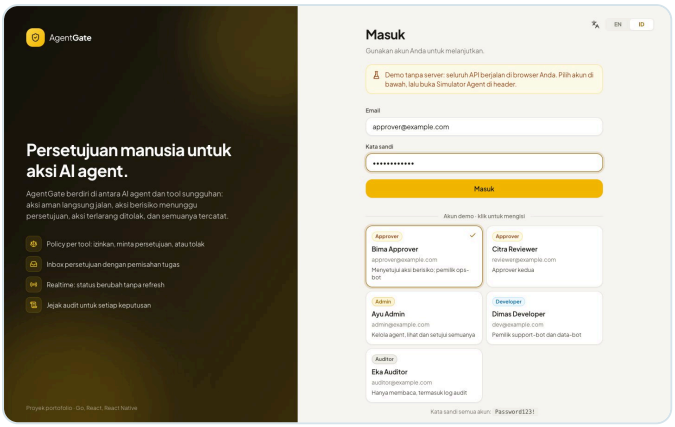
Backend	Go 1.25 · Gin · GORM + pgx · Viper · golang-jwt + bcrypt · go-i18n · zap · Swaggo (Swagger) · Testify + GoMock
Database	PostgreSQL 17 (Docker) · Unique (agent, Idempotency-Key) · Guarded status UPDATE
Web	React 18 · TypeScript · Vite · Redux Toolkit (RTK Query) · React Router · react-hook-form + yup · Tailwind CSS · Vitest · Vercel
Mobile	React Native 0.87 (New Architecture, Hermes) · React Navigation 7 · RTK Query · react-native-keychain

TECHNICAL DECISIONS

1. **Fail closed**
   If no rule matches, the tool's default effect applies. If a condition cannot be evaluated (wrong type or a broken regex), evaluation stops with REQUIRE_APPROVAL. A policy mistake never ends in ALLOW.
2. **What is approved is what runs**
   When a request arrives, a SHA-256 hash of the canonical JSON arguments is stored. Execution uses the stored arguments and re-checks the hash, so arguments cannot change after the approver has seen them.
3. **Race-safe decisions**
   Every status change is an UPDATE … WHERE id = ? AND status = ? inside one transaction with the timeline and audit rows. If two approvers click at once, exactly one succeeds and the other gets a 409; approving a call past its deadline is refused too.
4. **Idempotency and at-most-once side effects**
   The (agent, Idempotency-Key) pair is unique in the database, so concurrent duplicates create exactly one call; this is covered by integration tests against a real PostgreSQL. A call interrupted by a crash is never re-run, so a refund or deploy cannot happen twice.
5. **One set of test vectors for two engines**
   The policy engine exists twice: in Go for the backend and in TypeScript for the browser demo. Both run the same 39 test vectors from one JSON file, so the demo decides exactly like the server.
6. **Sensitive data masked everywhere**
   Fields marked sensitive (such as customer_email and email bodies) are replaced with *** in console responses, previews, and the audit log. Only the executor sees the real values.

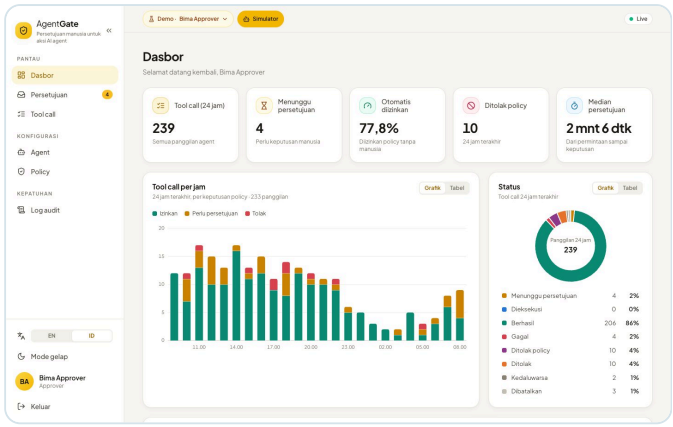
HOW TO USE IT

These steps follow the same order as the tutorial video.



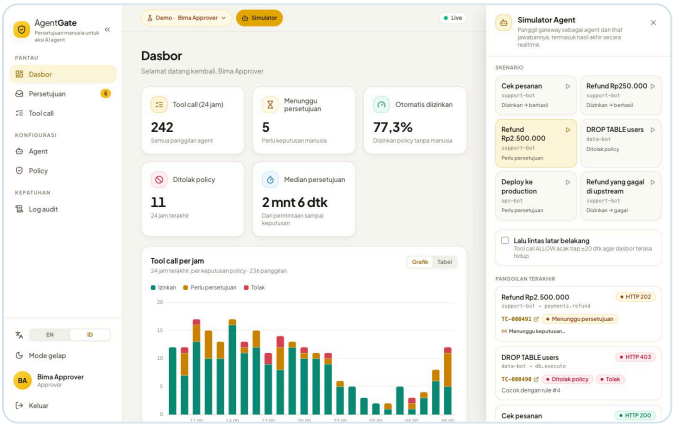
1 Sign in with a demo account

Open the demo, click the Bima Approver card to fill in the email and password, then click Sign in. There are five demo accounts, one per role (Approver, second Approver, Admin, Developer, Auditor), all with the password Password123!. The /?as=approver link signs you in as Bima directly.



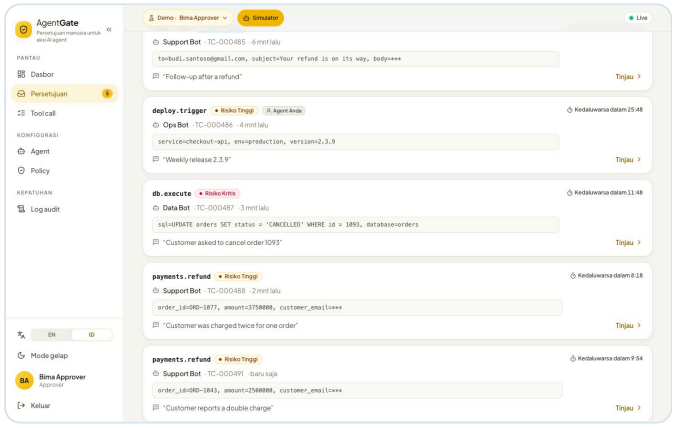
2 Read the dashboard

The cards at the top show 24-hour tool calls, calls awaiting approval, the share allowed automatically, policy denials, and median approval time. Below them are an hourly chart by policy decision and a status donut.



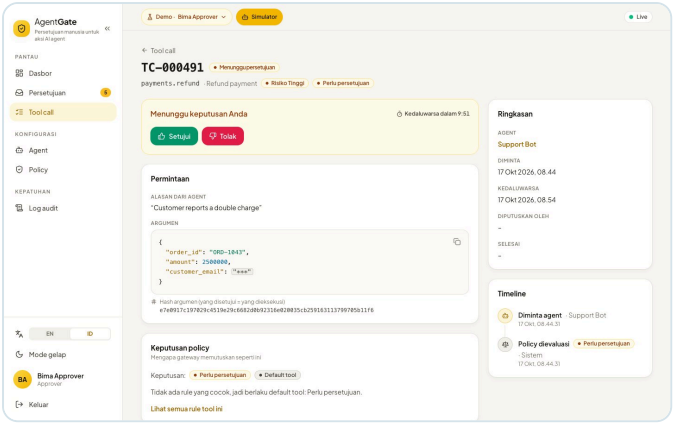
3 Call the gateway from the Simulator

Click Simulator in the header, then run Look up an order (HTTP 200, succeeds at once), DROP TABLE users (403, denied by rule #4), and Refund Rp2,500,000 (202, waiting for approval). The Approvals badge in the sidebar goes up without a refresh.



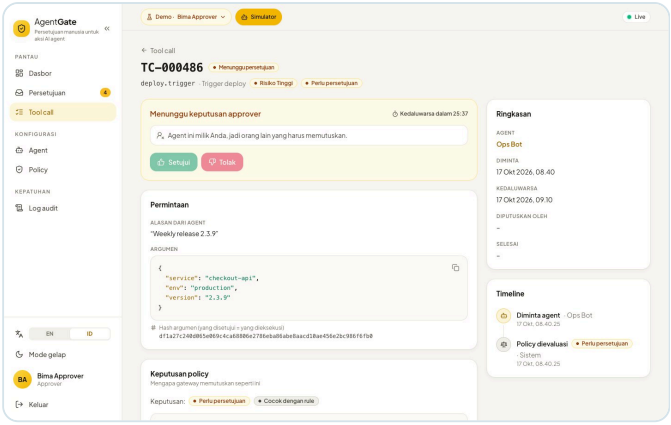
4 Open the approvals inbox

Close the simulator and open Approvals. The refund you just sent appears as "just now", with an expiry countdown, an argument summary, and the agent's reason.



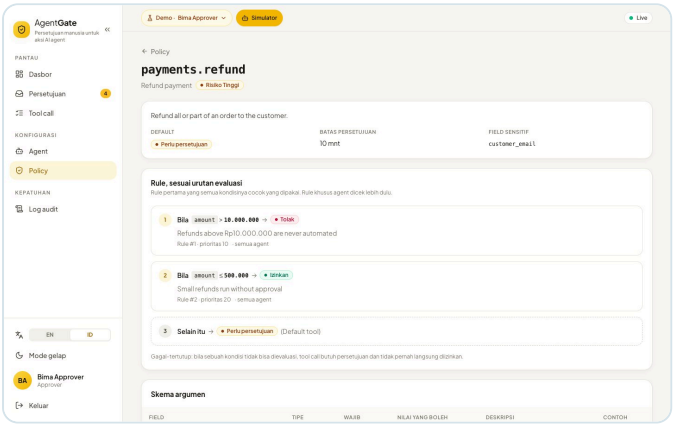
5 Review, then approve

Click Review. The detail page shows the arguments (customer_email masked), the argument hash, and why the policy decided: no rule matches Rp2,500,000, so the tool default (requires approval) applies. Click Approve, add a note, and confirm; the status becomes Succeeded and the Simulator panel receives the result too.



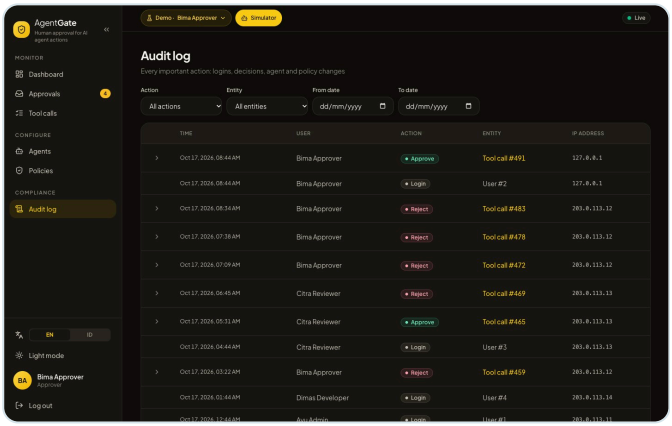
6 Try to approve your own agent's call

Go back to Approvals and open the deploy.trigger call labelled "Your agent". Bima owns ops-bot, so Approve and Reject are locked with a note that someone else has to decide. Switch persona to Citra Reviewer from the Demo badge to approve it.



7 Read a tool policy

Open Policies → payments.refund. Rules are shown in plain language in evaluation order: if amount > 10,000,000 → Deny, if amount ≤ 500,000 → Allow, otherwise approval is required. The tool's argument schema is listed below.



8 Check the audit log, theme, and language

Open Audit log to see the approval you just made, with time, user, and IP address. Click Dark mode and EN in the sidebar to switch theme and language. Reset demo data in the Demo badge menu restores the initial data.

LIMITATIONS & NOTES

- The live demo runs without a backend: data lives in each visitor's browser localStorage, and the gateway rules are reimplemented in TypeScript (tested with the same vectors as the Go backend).
- All six tools are sandboxes: they simulate 300–900 ms of latency with no real side effects. There is no executor that calls a real HTTP service yet.
- Policy rules are still seeded through SQL; the console can read policies but not edit them yet.
- The real-time hub is in memory, so the backend must run as a single instance. The mobile app needs a running backend and has not been published to the Play Store/App Store.

WHAT'S NEXT

- A policy editor with a dry-run simulator before rules are saved.
- Two approvers for Critical tools, and Android push notifications (FCM) for the approver app.
- An MCP adapter on top of the gateway and real HTTP executors for upstream services.